Universidad Nacional Autónoma de México
Facultad de Ciencias



Inteligencia Artificial

Laboratorio 1.4

Docente: *Dra. JessicaSarahiMéndezRincón*

Alumna: *OsorioLopezClaudiaYarinet*

Fecha de realización: 31/08/2026

Estructuras de datos en Python

Cada lenguaje de programación adapta las estructuras de datos globales a sus características propias, por lo cual, comprender algunos conceptos generales es fundamental para ampliar nuestro nivel de fluidez y abstracción al momento de aprender a programar en Python o en cualquier otro lenguaje.

Las matrices son una de las estructuras de datos fundamentales en el mundo de la programación, estas se encuentran disponibles en la mayoría de los lenguajes de códigos y tienen múltiples utilidades al momento de implementar diferentes algoritmos. Las matrices consisten en registros de datos de tamaño fijo que permiten que cada elemento se ubique de manera eficiente en función de su índice.

Las matrices almacenan la información en bloques de memoria adyacentes, por lo cual son consideradas estructuras de datos contiguas, en contraposición a otras como las listas enlazadas que vinculan las diferentes posiciones de los espacios de memoria que utilizan (Real Python, 2020).

Una analogía del mundo real para una estructura de datos matricial es un estacionamiento de vehículos. Si consideramos el estacionamiento como un todo, podemos tratarlo como un solo objeto. Dentro del estacionamiento podemos encontrar lugares indexados por un número único, estos lugares son contenedores para vehículos; cada espacio de estacionamiento puede estar vacío o tener un automóvil, una moto o algún otro vehículo estacionado en él (Real Python, 2020).

En la vida real es posible encontrar que todos los estacionamientos no son iguales, algunos pueden estar restringidos a un solo tipo de vehículo. Por ejemplo, el estacionamiento de camiones de transporte no permitiría que se estacionen bicicletas en él. Un estacionamiento restringido corresponde a una estructura de datos de matriz que permite solamente elementos que tienen el mismo tipo de datos almacenados en ellos.

Las matrices tienen un alto rendimiento, lo cual permite buscar rápidamente un elemento contenido dentro de la misma con solo conocer el índice de este. Según Real Python (2020), una implementación adecuada, garantiza un tiempo de acceso constante independientemente de la localización del elemento. Python incluye varias estructuras de datos con características similares a las matrices en su biblioteca estándar, cada una de las cuales tiene características ligeramente diferentes. A continuación, se describen algunas de estas.

Listas, tuplas y cadenas de caracteres

Listas

Las listas son parte del lenguaje central de Python y a pesar de su nombre, en el fondo se comportan como si fueran matrices dinámicas. Esto significa que una lista nos da la posibilidad de agregar o eliminar elementos, ajustando automáticamente el espacio que tiene asignado al almacenamiento de estos mediante la asignación o liberación de memoria.

Las listas de Python pueden contener cualquier tipo de elementos, ya que todos estos se consideran como objetos, incluidas las funciones. Por lo tanto, se pueden mezclar y combinar diferentes tipos de datos y almacenarlos todos en una sola lista. Esta puede ser

una característica significativa, pero la desventaja es que, al admitir múltiples tipos de datos al mismo tiempo, los datos generalmente están menos compactos. Como resultado, toda la estructura ocupa más espacio.

Según Matthes (2019), una lista es una colección de elementos ordenados de una forma particular. Una lista puede contener caracteres, cadenas de caracteres, valores numéricos o lógicos. Como buena práctica es recomendable asignarles nombres en plural, para así recordar fácilmente que su contenido está compuesto por elementos variados.

En el Tabla 1, se puede observar cómo se realiza la declaración de una lista donde todos sus elementos son del mismo tipo, y en la Tabla 2, tenemos un ejemplo de una lista con elementos variados:

Declaración de una lista de elementos iguales

```
estudiantes = ["Raul", "Alejandro", "Erick", "Amalia"]
```

Declaración de una lista de elementos variados

```
datos_estudiante = ["Raul", "Mexico", 176, 98.6, True]
```

Para acceder a un elemento dentro de una lista, se indica la posición de este mediante el uso de un índice, la sintaxis se compone por el nombre del elemento, procedido por el valor del índice, cerrado entre corchetes. Python considera que el primer elemento de la lista se encuentra en la posición 0, y no en la 1, como se pudiera deducir, esto se debe a la forma en se implementan las listas a bajo nivel.

```
datos_estudiante[0]
```

```
'Raul'
```

El segundo elemento de la lista está en la posición 1, por lo cual siguiendo este patrón es muy fácil determinar cuál es el valor adecuado de un índice para un elemento en específico

```
datos_estudiante[1]
```

```
'Mexico'
```

Operaciones con listas

En los lenguajes de programación orientados a objetos es muy común encontrar funciones específicas relacionadas con algún tipo de elemento, a ese tipo de función se les conoce como “métodos”. Python implementa varios métodos para realizar diversas operaciones con las listas. A continuación, se presentan algunos:

Método append(x)

Este método agrega un elemento al final de la lista.

```
estudiantes.append("Maria")
print(estudiantes)
```

```
['Raul', 'Alejandro', 'Erick', 'Amalia', 'Maria']
```

Método insert(i, x)

Este método agrega un elemento en un índice dado, el primer argumento indica el valor del índice

```
datos_estudiante.insert(2,"Nuevo Leon")
print(datos_estudiante)
```

```
['Raul', 'Mexico', 'Nuevo Leon', 176, 98.6, True]
```

Método remove(x)

Elimina el valor de la lista que coincide con el elemento indicado por x, en caso de no encontrarlo devuelve un error

```
datos_estudiante.remove(176)
print(datos_estudiante)
```

```
['Raul', 'Mexico', 'Nuevo Leon', 98.6, True]
```

Métodos pop([i])

Este método elimina un elemento de la lista que se encuentra en la posición indicada por el valor del índice i; en caso de no incluirlo, se procede a eliminar el último valor de la lista. En todas las ocasiones el método devuelve el contenido del valor eliminado. En la notación utilizada el uso de los corchetes dentro del paréntesis indica que la condición es opcional

```
datos_estudiante.pop(1)
'Mexico'
```

```
'Mexico'
```

Tuplas

Según Real Python (2020), una lista es adecuada para almacenar elementos que pueden cambiar durante el uso del programa. Esta propiedad es importante cuando trabajamos con aplicaciones que implican listas de usuarios o almacenar las características de un personaje dentro de un videojuego. Pero en ocasiones queremos crear listas que sabemos que no sufrirán ningún tipo de modificación, para este tipo de situaciones en Python encontramos las "Tuplas".

Al igual que las listas, las tuplas son parte fundamental del núcleo básico de Python. Sin embargo, a diferencia de las listas, las tuplas son inmutables. Esto significa que los elementos no se pueden agregar ni eliminar de forma dinámica; todos los elementos de una tupla deben definirse en el momento de su creación. Las tuplas también pueden contener tipos de datos arbitrarios. Tener esta flexibilidad es una ventaja, pero nuevamente, también significa que los datos están menos compactos y su estructura ocuparía mayor espacio.

Una tupla se define de una forma muy parecida a una lista, donde solamente se sustituirían los corchetes por paréntesis:

```
datos_pelicula = ("Tom_Cruise", "Misión_Imposible", 110, 1996, 4.7)
```

Una vez definida la tupla se puede acceder a sus elementos individuales mediante su índice correspondiente, exactamente como se haría con una lista

```
datos_pelicula[3]
```

```
1996
```

Por ejemplo, si quisiéramos definir una forma rectangular donde sabemos que sus dimensiones no van a variar durante la ejecución del programa la forma más adecuada sería la siguiente

```
rectangulo = (100, 250)
```

Arreglos

Una forma de almacenar variables de tipo numérico como los enteros o los números de punto flotante de una manera eficiente es mediante el uso de arreglos. Los arreglos son matrices con características mutables y que se comportan de manera similar a las listas excepto por una diferencia importante: están restringidas a un solo tipo de datos.

Debido a esta restricción, los objetos de tipo arreglo cuando contienen muchos elementos, son más eficientes en cuanto al espacio que las listas y tuplas. Los elementos almacenados en ellos están compactos y esto puede ser útil si necesita almacenar muchos elementos del mismo tipo. Python no soporta una estructura llamada "arreglo" de forma nativa, pero en su defecto utiliza las listas con igual tipo de variable como argumento.

Además, esta estructura admite muchos de los mismos métodos que las listas normales, por lo que, en caso necesario se puedan usar como un reemplazo directo sin requerir nuevos cambios en el código de la aplicación.

Un arreglo se definiría entonces de la misma forma que vimos en el tema de las listas

```
colores = ["rojo", "azul", "verde", "blanco", "amarillo"]
```

Una función muy útil que nos permite determinar rápidamente la longitud de un arreglo es len(). Funciones como len() o print() son internas del Python, y son conocidas como "funciones propias del lenguaje"

```
len(colores)
```

```
5
```

```
Comienza a programar o generar con IA.
```

Cadenas de caracteres

Python utiliza un objeto especial para almacenar datos de texto en forma de secuencia inmutable llamado: “cadena de caracteres”. De forma práctica esta estructura se puede considerar como una matriz inmutable de caracteres; curiosamente, también es considerada recursiva porque cada carácter de una cadena es, en sí mismo, una cadena de longitud 1. Los objetos de cadena ahorran espacio porque están empaquetados de forma compacta y se especializan en un solo tipo de datos según Python Software Foundation (s.f.).

Python trabaja con textos en formatos Unicode, el cual según Oracle (s.f.), es el estándar de codificación de caracteres universal utilizado para la representación de texto en sistemas computacionales. Unicode facilita un modo consistente de codificación de texto multilingüe y facilita el intercambio de archivos de texto internacionales.

Debido a que las cadenas son inmutables en Python, modificar una cadena requiere crear una copia modificada. El equivalente más cercano a una cadena mutable es almacenar caracteres individuales dentro de una lista de acuerdo con Python Software Foundation (s.f.). La tabla 14 muestra la forma en que podemos declarar y asignarle valores a este tipo de estructura.

```
nombre = "David"
```

Una cadena también pudiera contener caracteres vacíos

```
saludo = "Hola, mi nombre es:"
```

Operaciones con cadenas de caracteres

Al igual que vimos con las listas, existen operaciones y métodos específicos para trabajar con cadenas de caracteres, a continuación, recorreremos algunos de ellos:

Concatenar cadenas

Concatenar cadenas es muy útil para combinar diferentes elementos en estructuras más complejas, por ejemplo

```
saludo_con_nombre = saludo + " " + nombre
print(saludo_con_nombre)
```

```
Hola, mi nombre es: David
```

En el caso anterior se agregó el carácter espacio, para acomodar visualmente la presentación del resultado.

Método upper()

Este método permite transformar toda la cadena a caracteres en mayúsculas

```
nombre.upper()
```

```
'DAVID'
```

Método lower()

Realiza la operación contraria a upper(), y es muy útil cuando es necesario comparar cadenas sin considerar que los caracteres están en mayúsculas o combinados entre sí

```
descripcion = "PYTHON es un Lenguaje de Programacion muy Potente"
```

```
descripcion.lower()
```

```
'python es un lenguaje de programacion muy potente'
```

Método capitalize()

Capitalize es un método muy útil a la hora de manejar posibles errores de tipado al momento de recibir cadenas por parte del usuario, por ejemplo, en una aplicación donde se solicita la introducción de un apellido y el usuario por equivocación lo escribe todos en minúsculas, sin cumplir con las normas del lenguaje

```
apellido = "martinez"
apellido.capitalize()
```

```
'Martinez'
```

Método strip()

Este método al igual que el anterior nos ayuda a depurar errores en las cadenas recibidas, por ejemplo, si el usuario introduce una cadena con espacios en blanco, strip() nos ayuda a eliminar toda esa información innecesaria

```
apellido = "    martinez      "
apellido.strip()
```

```
'martinez'
```

Método replace()

Replace() nos permite cambiar los caracteres de una cadena por otros

```
descripcion = "PYTHON es un Lenguaje de Programacion muy Potente"
descripcion.replace('P', 'p')
```

```
'pYTHON es un Lenguaje de programacion muy potente'
```

Índices negativos

En Python podemos utilizar índices negativos para invertir el orden del conteo

```
descripcion = "PYTHON es un Lenguaje de Programacion muy Potente"
descripcion[-1]
```

```
'e'
```

Rebanadas

Como revisaste anteriormente los índices nos permiten seleccionar un carácter dentro de una cadena, utilizando este concepto con unas pequeñas modificaciones podemos obtener rebanadas o secciones de esta.

Según Python Software Foundation (s.f.), la sintaxis de una operación de rebanadas está compuesta por tres secciones opcionales, separadas por dos puntos y contenida entre corchetes. La primera sección indica el índice del primer elemento a seleccionar, la segunda muestra el índice del último elemento y la tercera se refiere a los pasos o saltos que ejecutará entre los elementos

```
descripcion[1:20:1]
```

```
'YTHON es un Lenguaj'
```

Como revisaste anteriormente los índices nos permiten seleccionar un carácter dentro de una cadena, utilizando este concepto con unas pequeñas modificaciones podemos obtener rebanadas o secciones de esta.

Según Python Software Foundation (s.f.), la sintaxis de una operación de rebanadas está compuesta por tres secciones opcionales, separadas por dos puntos y contenida entre corchetes. La primera sección indica el índice del primer elemento a seleccionar, la segunda muestra el índice del último elemento y la tercera se refiere a los pasos o saltos que ejecutará entre los elementos (Tabla 23 y tabla 24).

Haz doble clic (o ingresa) para editar

```
descripcion[1:20:2]
```

```
'YHNe nLnuj'
```

Diccionarios y conjuntos

Diccionarios

Otro tipo de estructura de datos muy popular en el lenguaje Python son los diccionarios, los cuales almacenan un número arbitrario de objetos, cada uno identificado por una clave única. Es común referenciar la clave de diccionario con el término llave, pero en la práctica

ambos términos tienen el mismo significado.

De forma general, un diccionario es una colección de elementos relacionado en forma llave-valor. Cada llave es conectada con un valor, y se utiliza como índice para acceder al contenido de este. El valor puede ser un número, una cadena, una lista e incluso otro diccionario. En efecto, se puede utilizar cualquier objeto del lenguaje como elemento valor dentro de un diccionario (Real Python, 2020).

Una analogía con el mundo real serían las agendas telefónicas, donde se asocia un nombre (llave) al número de contacto (valor). De igual forma que en la vida real, una persona pudiera tener más de un número asociado, pero solamente la identificamos por un nombre que se pueda reconocer fácilmente. Al momento de localizar un dato dentro de los contactos, saltar a su nombre y rápidamente recuperas el valor deseado.

Los diccionarios también se denominan a menudo: mapas, hashmaps, tablas de búsqueda o matrices asociativas. Dentro de estos se permiten las operaciones de búsqueda, inserción y eliminación de cualquier objeto asociado con una clave determinada. Los diccionarios nos permiten encontrar rápidamente la información asociada con una clave determinada.

En Python, podemos utilizar el constructor dict() para crear esta estructura de datos, donde la declaración del diccionario se encierra entre corchetes y para las series de llave-valor se utilizan los paréntesis (Tabla 25).

```
estacionamiento = dict([('espacio_1', 'Nissan'),( 'espacio_2', 'Toyota'),( 'espacio_3', 'BMW')])
```

Una forma más organizada de declarar el mismo diccionario pudiera ser

```
estacionamiento = {
    'espacio_1': 'Nissan',
    'espacio_2': 'Toyota',
    'espacio_3': 'BMW',
}
```

Existen algunas restricciones sobre qué objetos se pueden usar como claves válidas. Los diccionarios de Python están indexados por claves que pueden tener cualquier valor al calcularle su función hash correspondiente, pero estos elementos deben ser estrictamente hashables. Según Donohue (2021), la función hash es un algoritmo matemático que convierte cualquier bloque de datos en una nueva serie de caracteres con una dimensión fija. De forma independiente de la longitud de los datos de entrada, el valor hash de salida tendrá siempre la misma longitud.

Un objeto hashable es un elemento cuya función hash nunca varía durante la ejecución del programa. Objetos inmutables, como las cadenas y los números funcionan muy bien como claves de diccionario. También pueden usar tuplas para esta función, siempre que estas sean hashables (Donohue, 2021).

Para acceder a un elemento dentro de un diccionario la forma correcta es mediante su respectiva llave

```
estacionamiento['espacio_2']
```

```
'Toyota'
```

Operaciones con diccionarios

Además de la búsqueda, se pueden realizar otras acciones básicas con los diccionarios.

1. Insertar elementos: para insertar un elemento es necesario indicar una nueva llave

```
estacionamiento['espacio_4'] = 'Mercedes'
```

2. Actualizar un elemento: un elemento se puede actualizar accediendo directamente a él mediante su llave respectiva

```
estacionamiento['espacio_3'] = 'Ford'
```

3. Eliminar elemento: para eliminar un elemento de un diccionario debemos utilizar la palabra reservada “del”, indicando de igual forma que en los ejemplos anteriores el valor mediante su llave

```
del estacionamiento['espacio_2']
```

Para la mayoría de los casos de uso, la implementación del diccionario integrado de Python es más que suficiente. Los diccionarios están altamente optimizados y son la base de muchas partes del lenguaje. Por ejemplo, todos los atributos de una clase y las variables de una

pila se almacenan internamente en diccionarios.

Los diccionarios de Python se basan en una implementación de tabla hash bien probada y finamente ajustada que proporciona las características de rendimiento necesarias para realizar rápidamente las operaciones de búsqueda, inserción, actualización y eliminación, en el caso necesario. Sin embargo, es importante mencionar que existen otras formas especializadas de diccionarios de terceros, como las listas de omisión o los diccionarios basados en árboles.

Conjuntos

Un conjunto es una colección desordenada de objetos que no permite elementos duplicados. Normalmente, esta estructura de datos se utiliza para probar rápidamente si un valor pertenece al conjunto, para insertar o eliminar nuevos valores en un conjunto y para calcular la unión o intersección de dos conjuntos.

Según Real Python (2020), en una implementación de conjunto adecuada, se espera que las pruebas de pertenencia se ejecuten rápidamente. Las operaciones de unión, intersección, diferencia y subconjunto suelen tomar un poco más de tiempo. Las implementaciones de conjunto incluidas en la biblioteca estándar de Python siguen estas características de rendimiento.

Al igual que los diccionarios, los conjuntos reciben un tratamiento especial en Python y tienen una sintaxis específica que los distingue de otras estructuras de datos del lenguaje. Por ejemplo, para crear un conjunto se utilizan las llaves, a diferencia de las tupas y listas que se declaran con paréntesis o corchetes respectivamente

```
conjunto_A = {'a', 'x', 'r', 'g', 'f', 'l', 'n', 'o', 'r', 'p'}
```

También se puede generar un conjunto utilizando la función set() sobre objetos iterables; si no se agrega ningún elemento se crea un conjunto vacío. Para generar conjuntos vacíos no se debe usar la sintaxis de doble llaves {}; pues este es un uso ambiguo que genera un diccionario nulo en vez del conjunto deseado

```
conjunto_B = set("murcielago")
```

Entre las operaciones que podemos hacer con los conjuntos están:

1. Prueba de pertenencia: determina si el elemento pertenece al conjunto

```
'a'in conjunto_B
```

```
True
```

2. Unión: devuelve los elementos que pertenecen a ambos conjuntos

```
conjunto_A | conjunto_B
```

```
{'a', 'c', 'e', 'f', 'g', 'i', 'l', 'm', 'n', 'o', 'p', 'r', 'u', 'x'}
```

3. Intersección: entrega los elementos que coinciden en ambos conjuntos

```
conjunto_A & conjunto_B
```

```
{'a', 'g', 'l', 'o', 'r'}
```

4. Diferencia: esta operación entrega los elementos que pertenecen al primer conjunto pero que no están en el segundo

```
conjunto_A - conjunto_B
```

```
{'f', 'n', 'p', 'x'}
```

5. Complemento: devuelve todos los elementos que pertenecen a cada conjunto de forma exclusiva

```
conjunto_A ^ conjunto_B
```

```
{'c', 'e', 'f', 'i', 'm', 'n', 'p', 'u', 'x'}
```

Python y su biblioteca estándar proporcionan varias implementaciones de conjuntos. Echemos un vistazo a ellos.

Conjuntos de referencia: Son los conjuntos generados con la función set(), los cuales son mutables, ya que permiten la inserción y eliminación dinámica de elementos. Estos conjuntos tienen las mismas características de rendimiento que los diccionarios y pueden almacenar objetos que generan cualquier tipo de hash. Conjuntos inmutables: Otra función propia del lenguaje que permite generar conjuntos es frozenset(), la característica principal de estos objetos es que son estáticos y solo permiten operaciones de consulta en sus elementos, no permite inserciones o eliminaciones. Debido a que estos conjuntos son estáticos, se pueden usar como claves de diccionario o como elementos de otro conjunto, algo que no es posible con conjuntos de referencia regulares (mutables). Otras estructuras de datos

Pilas (LIFO)

La pila es una estructura de datos lineal que sigue un orden particular en el que se realizan las operaciones, en este caso mantienen un comportamiento de "ultimo en entrar/ primero en salir", cada vez que se agregan o eliminan objetos a la estructura. A diferencia de las listas o matrices, las pilas normalmente no permiten el acceso aleatorio a los objetos que contienen. Las operaciones de inserción y eliminación también se denominan a menudo push y pop (Matthes, 2019)

Colas (FIFO)

Una cola es una colección de objetos que tiene un orden específico, en este caso se utiliza la filosofía "primero en entrar/primero en salir" para las operaciones de inserción y eliminación de elementos, estas operaciones normalmente se denominan poner en cola y sacar de cola. A diferencia de las listas o matrices, las colas normalmente no permiten el acceso aleatorio a los objetos que contienen (Matthes, 2019).

Sentencias condicionales y de control de flujo (bucles) en Python

A veces, también es necesario repetir lo que hace un programa más de una vez. En programación a eso se le conoce como bucle, y en Python los dos mecanismos usados para ello son for y while. En este tema verás múltiples ejemplos en los que ambos se aplican a problemas y también objetos, tales como listas y diccionarios.

Uso de las sentencias condicionales (If, if-else y if-elif)

Matthes explica que el propósito de la sentencia if es evaluar si el valor de una expresión es Verdadero (True) o Falso (False) y a esto se le conoce como prueba (test) condicional. Python usa ese tipo de valores para decidir si el código acotado por la instrucción if debe ejecutarse o no.

Si el test condicional resulta verdadero, Python ejecuta el código que sigue a la instrucción if, por el contrario, si el test resulta falso, Python ignora el código que sigue a la instrucción if.

La mayoría de los tests condicionales comparan el valor actual de una variable con un valor de interés específico. El primer ejemplo consiste en comparar el valor de la variable color:

```
color = 'rojo'
color == 'rojo'
```

```
True
```

La línea 1 del código establece el valor de color en 'rojo' usando un solo signo igual (=). La línea 2 verifica si el valor de color es 'rojo' usando un doble signo igual (==), este operador de igualdad devuelve True si los valores de cada lado del operador, izquierdo y derecho, coinciden, y False en caso contrario. Los valores de este ejemplo coinciden, por lo que Python devuelve True.

Cuando el valor de color es distinto de 'rojo', el test devuelve False:

```
color = 'negro'
color == 'rojo'
```

```
False
```

Como puede notarse entonces que el operador (=) significa una declaración, por lo que la línea 1 podría leerse como "Establecer el valor de color como 'negro'. Mientras que el doble signo de igual (==) de la línea 2 representaría la pregunta: "¿El valor de color es 'rojo'?" La mayoría de los lenguajes de programación emplean estos operadores de la misma manera.

Cuando se desea determinar si dos valores no son iguales, el signo de igual debe combinarse con un signo de exclamación (!=). Al igual que en otros lenguajes de programación, el signo de exclamación representa no.

A continuación, se utiliza la sentencia if para examinar el uso del operador de desigualdad (!=). El objetivo es almacenar los ingredientes del pedido de una ensalada e imprimir un mensaje si la persona no incluyó tomate:

```
ingrediente = 'lechuga'
if ingrediente != 'tomate':
  print("Sin tomate")
```

```
Sin tomate
```

En este caso, la línea 2 compara el valor de ingrediente con el valor de 'tomate'. Como estos dos valores no coinciden, entonces Python devuelve True y ejecuta el código que sigue después de la instrucción if. Si ambos valores coincidieran, Python devolvería False y no imprimiría el argumento de la función print.

La mayoría de los tests condicionales buscan probar la igualdad (==), no obstante, en algunas ocasiones resultará más eficiente trabajar con la desigualdad (!=).

Por otra parte, también es posible probar la igualdad o desigualdad entre valores numéricos. Por ejemplo, el siguiente código verifica si dos números no son iguales e imprime un mensaje si la respuesta dada no es la correcta:

```
respuesta = 15
if ingrediente != '25':
  print("Esa no es la respuesta correcta. Por favor, intente de nuevo.")
```

```
Esa no es la respuesta correcta. Por favor, intente de nuevo.
```

En este test, el programa ejecuta la línea 1 porque el valor de la respuesta (15) no es igual a 25, imprimiendo el mensaje que contiene la función print.

Existe otro tipo de situaciones en las que se requiere que dos condiciones se cumplan para realizar una acción o que solo una de ellas sea suficiente. En estos casos, los operadores lógicos and y or son muy útiles.

El operador and se utiliza para comprobar si dos condiciones son verdaderas al mismo tiempo, cuando ambas se cumplen la expresión general del if se considera True. Pero en cambio, si alguna o ambas condiciones fallan, el resultado sería False. Por ejemplo, se desea verificar que dos personas tienen más de 18 años, el programa sería el siguiente:

```
edad_0 = 20
edad_1 = 17
edad_0 >= 18 and edad_1 >= 18
```

```
False
```

```
edad_1 = 19
edad_0 >= 18 and edad_1 >= 18
```

```
True
```

En la línea 1, se definen dos edades, edad_0 y edad_1. Después, en la línea 2 se comprueba si ambas edades corresponden a 18 años o más. La condición del lado izquierdo del operador pasa, pero la del lado derecho falla, por lo que el resultado es False.

En la línea 4, se cambia el valor de edad_1 por 19. Dado que ahora es mayor que 18, ambas condiciones se cumplen y el resultado es True.

Para una mejor legibilidad, se pueden usar paréntesis alrededor de las condiciones a tratar, pero sin ser obligatorios. Si usan paréntesis, esa línea de código quedaría de la siguiente manera:

```
(edad_0 >= 18) and (edad_1 >= 18)
```

```
True
```

De modo similar, el operador or también permite verificar múltiples condiciones, pero en su caso, el test se valida como True cuando una o ambas condiciones se cumplen. Y solo cuando ambas condiciones fallan, el resultado es False.

Si en el ejemplo anterior, fuese suficiente que solo una de las dos edades fuese mayor a 18 años, el programa y resultado sería el siguiente:

```
edad_0 = 20
edad_1 = 17
edad_0 >= 18 or edad_1 >= 18
```

```
True
```

```
edad_1 = 19
edad_0 >= 18 and edad_1 >= 18
```

```
True
```

Una vez comprendidas este tipo de pruebas condicionales, es posible elaborar declaraciones if más complejas. Existen varios tipos diferentes de esta y la elección de cuál usar depende de la cantidad y el modo en que las condiciones que se necesitan probar.

Como se vio en algunos ejemplos al inicio de este tema, el tipo de declaración if más simple consiste en una prueba condicional y una acción:

if prueba_condicional: Hacer algo

Sin embargo, muchas veces se requiere tomar una acción cuando se cumple una condición y otra diferente para todos los demás casos, entonces se utiliza la sintaxis if-else de Python.

Un bloque if-else es similar a una instrucción if simple, pero la instrucción else permite definir una acción o un conjunto de acciones que se ejecutan cuando una condición no se cumple.

Por ejemplo, el siguiente programa detecta si una persona tiene la edad suficiente para votar o no:

```
edad = 17
if edad >= 18:
  print("Tienes la edad suficiente para votar")
else:
  print("Lo sentimos, eres demasiado joven para votar")
```

```
Lo sentimos, eres demasiado joven para votar
```

Si la condición de la línea 2 se cumple, entonces se ejecuta el print de la línea 3. Pero si la condición resulta False, entonces se ejecuta lo que está después del else de la línea 4. Debido a que la edad es menor que 18, el resultado es el mensaje del else.

El código anterior funciona porque solo tiene dos situaciones posibles: una persona tiene la edad suficiente para votar o no. La estructura if-else funciona bien para situaciones como esta.

No obstante, a menudo, se requieren probar más de dos situaciones posibles. Para casos como estos es necesario usar la sintaxis if-elif-else. Un ejemplo sería un museo en el que se cobra diferentes tarifas para diferentes grupos de edad:

La entrada para cualquier persona menor de 5 años es gratuita. Cualquier persona entre las edades de 5 y 18 paga $95.

Cualquier persona mayor de 18 años paga $80.

Cualquier persona de 60 años o más, paga $40, la mitad de la entrada regular.

Un programa que resuelve el caso anterior es el siguiente:

```
edad = 11
if edad < 5:
    precio = 0
elif edad < 18:
    precio = 95
elif edad < 60:
    precio = 80
else:
    precio = 40

print (f"Su costo de admisión es {precio}.")
```

```
Su costo de admisión es 95.
```

Se pueden emplear tantos bloques elif como sea necesario. Por ejemplo, si el museo implementara un descuento para otro tipo de persona, podría agregarse una prueba condicional más al código para determinar si alguien califica para dicho descuento.

if-elif-else es una herramienta poderosa, sin embargo, solo es apropiado usarla cuando necesita solo una de todas las condiciones puede suceder, ya que tan pronto como Python encuentra cual cumple, el resto se omite. Por lo que cuando se requiere comprobar todas las condiciones, se debe utilizar una serie de sentencias if una tras otra sin bloques elif o else. Ese tipo de técnica tiene sentido cuando más de una condición puede ser verdadera.

Uso de ciclos (for y while)

El iterador for funciona tomando una colección de elementos y ejecutando un bloque de código una vez para cada elemento de la colección. En contraste, el tipo de bucle while se ejecuta siempre que cierta condición sea verdadera. A continuación, se abordará el uso e implementación del segundo y después se verán ejemplos para el primero.

El siguiente ejemplo es un programa con un bucle while que sirve para contar del 1 a 5:

```
número_actual = 1          # Empezamos en 1
while número_actual <= 5:  # Mientras sea menor o igual a 5
    print(número_actual)
    número_actual += 1     # Sumamos 1
```

```
1
2
3
4
5
```

```
número_actual = 20
while número_actual >= 5:  # Mientras sea mayor a 5
    print(número_actual)
    número_actual -= 1     # Restamos 1
```

```
20
19
18
17
16
15
14
13
12
11
10
9
8
7
6
5
```

Lo más probable es que los programas que utilizas a diario contienen este tipo de bucles. Por ejemplo, un juego necesita uno para seguir funcionando todo el tiempo que se esté usando y solo se detiene hasta que le solicitas salir.

Los programas dejarían de ser divertidos si pararan antes de que se les indique o si continuaran ejecutándose incluso después de que el usuario desee salir. Por esta razón, los bucles while son muy útiles.

Pero lo anterior, llevaría a realizarse la siguiente pregunta ¿cómo puede hacerse un programa en que el usuario decida cuando salir? A continuación, se realiza un ejemplo para ello:

```
aviso = "\nDime algo y te lo repetiré:"
aviso += "\nIntroduce 'salir' para finalizar el programa "
mensaje = ""
while mensaje != 'salir':
    mensaje = input(aviso)
    print(mensaje)
```

```
Dime algo y te lo repetiré:
Introduce 'salir' para finalizar el programa fapalapachala
fapalapachala

Dime algo y te lo repetiré:
Introduce 'salir' para finalizar el programa salir
salir
```

En la línea 1 y 2, se define un aviso que le indica al usuario dos opciones: ingresar un mensaje o ingresar el valor de salida (en este caso, 'salir'). Después, en la línea 2 se configura la variable mensaje, cuya función es detectar cualquier valor que ingrese el usuario. En la línea 3 se le asigna el valor inicial de "" y entonces se vuelve una cadena vacía.

Cuando Python llega a la instrucción while en la línea 4 por primera vez, el valor de mensaje se compara con 'salir', pero aún no se ha ingresado ninguna entrada del usuario y entonces el programa entra al bucle.

En la línea 5, Python muestra el aviso y espera a que el usuario ingrese su entrada y así asignarlo a mensaje. Entonces cuando el programa pasa a la línea 6, el mensaje se imprime y Python volverá a evaluar la condición del bucle while. Siempre que el usuario no haya

ingresado la palabra 'salir', el mensaje que introduzca se desplegará. Cuando el usuario finalmente introduzca 'salir', el programa saldrá del bucle y como no hay más instrucciones finalizará.

Lubanovic (2019) explica que Python hace un uso frecuente del iterador for por una buena razón. Le hace posible atravesar estructuras de datos sin importar qué tan grandes sean o la manera en que estén implementadas. Incluso puede iterar sobre datos que se van creando sobre la marcha, esto permite al programa procesar paquetes de datos que, de otro modo, no cabrían en la memoria de la computadora en una sola ejecución.

En Python sería posible realizar el siguiente programa empleando while:

```
conejos = ['Oddy', 'Tiky', 'Manchas', 'Nubito']
actual = 0
while actual < len(conejos):
    print(conejos[actual])
    actual += 1
```

```
Oddy
Tiky
Manchas
Nubito
```

Pero de acuerdo con Lubanovic, sería más conveniente hacer lo mismo de la siguiente manera usando el iterador for:

```
conejos = ['Oddy', 'Tiky', 'Manchas', 'Nubito']
actual = 0
for conejo in conejos:
    print(conejo)
```

```
Oddy
Tiky
Manchas
Nubito
```

Las listas como la del ejemplo anterior, conejos, son parte de los objetos iterables de Python, junto con cadenas (strings), tuplas, diccionarios, conjuntos y algunos más. La iteración de tuplas o listas se produce con un elemento a la vez, mientras que para cadenas un carácter a la vez, como se muestra a continuación:

```
palabra = 'caja'
for letra in palabra:
    print(letra)
```

```
c
a
j
a
```

Cuando se itera un diccionario se obtienen sus claves (keys). En este ejemplo, las claves son los datos de un usuario en una red social:

```
usuario =  {'nombre': 'Efraín', 'correo electrónico': 'efrain_vm@gmail.com',
'fecha de nacimiento': '15-08-2004'}
for dato in usuario:  #  ó for dato in usuario.keys():
    print(dato)
```

```
nombre
correo electrónico
fecha de nacimiento
```

Para iterar sobre los valores en lugar de las claves, usa la función values() del diccionario:

```
for valor in usuario.values():
  print(valor)
```

```
Efraín
efrain_vm@gmail.com
15-08-2004
```

Para devolver la clave y el valor en una tupla, puede usarse la función items():

```
for artículo in usuario.items():
    print(artículo)
```

```
('nombre', 'Efraín')
('correo electrónico', 'efrain_vm@gmail.com')
('fecha de nacimiento', '15-08-2004')
```

También se puede asignar una tupla en un solo paso. Para cada tupla devuelta por items(), se asigna el primer valor (la clave) a dato y el segundo (el valor) a contenido:

```
for artículo in usuario.items():
    print(artículo)
for dato, contenido in usuario.items():
    print('El dato de', dato, 'contiene', contenido)
```

```
('nombre', 'Efraín')
('correo electrónico', 'efrain_vm@gmail.com')
('fecha de nacimiento', '15-08-2004')
El dato de nombre contiene Efraín
El dato de correo electrónico contiene efrain_vm@gmail.com
El dato de fecha de nacimiento contiene 15-08-2004
```

La sentencia break en un bucle for le permite salir del mismo, de la misma manera en que lo hace para un bucle while. Por otra parte, si se inserta un continue en un bucle for, el programa salta a la siguiente iteración del bucle.

Similar al while, el for cuenta con un else opcional que comprueba si se completó. Si no se usa un break, entonces la instrucción else se ejecuta.

Lo anterior es útil cuando se desea verificar que el ciclo previo for se ejecutó hasta su finalización, en vez de detenerse antes con un break. A continuación, se muestra un ejemplo de cómo un bucle for imprime el nombre del queso si este se encuentra en la tienda y después de ello el bucle “se rompe” a través del break:

```
quesos = []
for queso in quesos:
    print('Esta tienda tiene un estupendo queso ', queso)
    break
else:  # si el break no se ejecutó significa que no hay queso
    print('Esta tienda no tiene ni un queso.')
```

```
Esta tienda no tiene ni un queso.
```

Matthes, por su parte agrega que, aunque el bucle for es eficaz para recorrer una lista, ésta no debe modificarse mientras el bucle está operando porque Python tendría problemas al realizar el seguimiento de sus elementos. Para modificar una lista mientras se trabaja con ella, Matthes recomienda usar un bucle while

Desafio

Parte 1

Utilizando la herramienta Júpiter Notebook programa lo siguiente:

- Escribe una lista que contenga un mínimo de cinco elementos.
- Agrega un nuevo elemento en el índice 2.
- Agrega un elemento al final de la lista.
- Escribe una cadena de caracteres que contenga tres palabras.
- Cuenta los caracteres de la cadena generada.
- Guarda la cadena resultante toda en mayúsculas en una nueva variable.
- Escribe un diccionario que contenga al menos cuatro elementos y cuyos índices coincidan con las primeras letras del alfabeto.
- Agrega un nuevo elemento al diccionario.
- Elimina el elemento del índice “c”.
- Cuenta los elementos que componen el diccionario después de completadas las operaciones anteriores.

Parte 2

Con lo aprendido en clase, de cada uno implementa un programa en Python con la solución:

- Una sala de cine cobra diferentes precios de entrada según la edad de la persona. Si una persona es menor de 3 años, la entrada es gratuita; si tienen entre 3 y 12, el boleto es de 10 pesos ; y si son mayores de 12 años, el boleto cuesta $ 15. Escribe un bucle en el que se solicite a los usuarios su edad y despliegue el costo de la entrada.
- Crea un diccionario que contenga la información de una canción (artista, álbum, duración y género) y despliega el contenido de cada elemento (keys, values y tuplas)

Ejercicio 1

- Escribe una lista que contenga un mínimo de cinco elementos.
- Agrega un nuevo elemento en el índice 2.
- Agrega un elemento al final de la lista.

```
# Crear una lista con al menos cinco elementos
equipos = ["Pumas", "Guadalajara", "America", "Cruz Azul", "Monterrey", "Atlas"]

# Imprimir la lista original
print("Lista original:")
print(equipos)

# Agregar un nuevo elemento en el índice 2
nuevo_elemento = "Puebla"
equipos.insert(2, nuevo_elemento)

# Imprimir la lista después de agregar el nuevo elemento en el índice 2
print("\nLista después de agregar un nuevo elemento en el índice 2:")
print(equipos)

# Agregar un elemento al final de la lista
elemento_final = "Santos"
equipos.append(elemento_final)

# Imprimir la lista después de agregar un elemento al final
print("\nLista después de agregar un elemento al final:")
print(equipos)
```

```
Lista original:
['Pumas', 'Guadalajara', 'America', 'Cruz Azul', 'Monterrey', 'Atlas']

Lista después de agregar un nuevo elemento en el índice 2:
['Pumas', 'Guadalajara', 'Puebla', 'America', 'Cruz Azul', 'Monterrey', 'Atlas']

Lista después de agregar un elemento al final:
['Pumas', 'Guadalajara', 'Puebla', 'America', 'Cruz Azul', 'Monterrey', 'Atlas', 'Santos']
```

Ejercicio 2

- Escribe una cadena de caracteres que contenga tres palabras.
- Cuenta los caracteres de la cadena generada.
- Guarda la cadena resultante toda en mayúsculas en una nueva variable.

```
# Escribe una cadena de caracteres que contenga tres palabras
saludo = "Hola Mi Mundo"

# Cuenta los caracteres de la cadena generada
caracteres = len(saludo)

# Imprime el número de caracteres
print("Número de caracteres:", caracteres)

# Guarda la cadena resultante toda en mayúsculas en una nueva variable
saludo_mayusculas = saludo.upper()

# Imprime la cadena en mayúsculas
print("Cadena en mayúsculas:", saludo_mayusculas)
```

```
Número de caracteres: 13
Cadena en mayúsculas: HOLA MI MUNDO
```

Ejercicio 3

- Escribe un diccionario que contenga al menos cuatro elementos y cuyos índices coincidan con las primeras letras del alfabeto.
- Agrega un nuevo elemento al diccionario.
- Elimina el elemento del índice “c”.
- Cuenta los elementos que componen el diccionario después de completadas las operaciones anteriores.

```
# Escribe un diccionario que contenga al menos cuatro elementos
diccionario = {
    'a': 'Argentina',
    'b': 'Brazil',
    'c': 'Cánada',
    'd': 'Dinamarca',
}

# Imprime el diccionario original
print("Diccionario original:")
print(diccionario)

# Agrega un nuevo elemento al diccionario
diccionario['e'] = 'España'

# Imprime el diccionario después de agregar un nuevo elemento
print("\nDiccionario después de agregar un nuevo elemento:")
print(diccionario)

# Elimina el elemento del índice "c"
diccionario.pop('c')

# Imprime el diccionario después de eliminar el elemento 'c'
print("\nDiccionario después de eliminar el elemento 'c':")
print(diccionario)

# Cuenta los elementos que componen el diccionario después de completadas las operaciones anteriores
elementos = len(diccionario)
print("\nNúmero de elementos en el diccionario después de las operaciones:")
print(elementos)
```

```
Diccionario original:
{'a': 'Argentina', 'b': 'Brazil', 'c': 'Cánada', 'd': 'Dinamarca'}

Diccionario después de agregar un nuevo elemento:
{'a': 'Argentina', 'b': 'Brazil', 'c': 'Cánada', 'd': 'Dinamarca', 'e': 'España'}

Diccionario después de eliminar el elemento 'c':
{'a': 'Argentina', 'b': 'Brazil', 'd': 'Dinamarca', 'e': 'España'}

Número de elementos en el diccionario después de las operaciones:
4
```

Problema 1.

Una sala de cine cobra diferentes precios de entrada según la edad de la persona. Si una persona es menor de 3 años, la entrada es gratuita; si tienen entre 3 y 12, el boleto es de $10; y si son mayores de 12 años, el boleto cuesta$ 15. Escribe un bucle en el que se solicite a

los usuarios su edad y despliegue el costo de la entrada.

```
# Solicitar la edad del usuario
edad = int(input("Ingresa tu edad: "))

# Calcular el costo de la entrada según la edad
if edad < 3:
    boleto = 0
elif 3 <= edad <= 12:
    boleto = 10
else:
    boleto = 15

# Mostrar el costo de la entrada al usuario
print(f"El costo de la entrada es de ${boleto}.")
```

```
Ingresa tu edad: 26
El costo de la entrada es de $15.
```

Problema 2.

Crea un diccionario que contenga la información de una canción (artista, álbum, duración y género) y despliega el contenido de cada elemento (keys, values y tuplas)

```
# Crear un diccionario con la información de la canción
cancion = {
    "artista": "Taylor Swift",
    "album": "Cancelled",
    "duracion": "3:36",
    "genero": "Pop"
}

# Mostrar las keys del diccionario
print("Keys:")
print(cancion.keys())

# Mostrar los values del diccionario
print("Values:")
print(cancion.values())

# Mostrar las tuplas (key, value) del diccionario
print("Tuplas:")
print(cancion.items())
```

```
Keys:
dict_keys(['artista', 'album', 'duracion', 'genero'])
Values:
dict_values(['Taylor Swift', 'Cancelled', '3:36', 'Pop'])
Tuplas:
dict_items([('artista', 'Taylor Swift'), ('album', 'Cancelled'), ('duracion', '3:36'), ('genero', 'Pop')])
```

Cerrar

```
# Conclusiones

Me gusto la practica, en particular la iteracion de letras, disfru
estos ejercicios. Me han ayudado muchisimo para poner a prueba mi
programacion, pero debo seguir practicando, porque pierdo practica

Tambien en el ejercicio sobre guardar canciones, pienso constantem
metadatos de las imagenes y canciones y como esto nos ayudara en e
```

Conclusiones

Me gusto la practica, en particular la iteracion de letras, disfruto bastante estos ejercicios. Me han ayudado muchisimo para poner a prueba mi capacidad de programacion, pero debo seguir practicando, porque pierdo practica.

Tambien en el ejercicio sobre guardar canciones, pienso constantemente en los metadatos de las imagenes y canciones y como esto nos ayudara en el futuro